UNIVERSIDADE TUIUTI DO PARANÁ
FACULDADE DE CIÊNCIAS EXATAS E DE TECNOLOGIA
CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

FABRICIO TITO COSTA GOULART
MATHEUS SERRUTE BAILO
MIGUEL DE CARLO GOMES
QUIZITO CRISTIANO AGOSTINHO
VITOR DA SILVA CAMPOS
ZACK REYEL FERREIRA BARROZO

WATCHFLOW: PLATAFORMA DE OBSERVABILIDADE, TELEMETRIA DE ATIVOS E MONITORAMENTO DE DESEMPENHO (APM)

Relatório Técnico-Científico e Especificação de Projeto de Software

CURITIBA
2026

FABRICIO TITO COSTA GOULART
MATHEUS SERRUTE BAILO
MIGUEL DE CARLO GOMES
QUIZITO CRISTIANO AGOSTINHO
VITOR DA SILVA CAMPOS
ZACK REYEL FERREIRA BARROZO

WATCHFLOW: PLATAFORMA DE OBSERVABILIDADE, TELEMETRIA DE ATIVOS E MONITORAMENTO DE DESEMPENHO (APM)

Relatório técnico-científico e de projeto de software apresentado à Faculdade de Ciências Exatas e de Tecnologia da Universidade Tuiuti do Paraná, como requisito avaliativo do 2º bimestre na disciplina de Projeto Interdisciplinar: Monitoramento de Sistemas do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas.

Professor Orientador: Prof. Me. André Luiz de Souza Paula

CURITIBA
2026

RESUMO

O presente trabalho apresenta a especificação técnica, a arquitetura e os resultados práticos da validação do **WatchFlow**, uma plataforma de monitoramento e observabilidade voltada à inspeção contínua de hardware e rastreamento de desempenho de aplicações (APM – *Application Performance Monitoring*). A indisponibilidade de sistemas e o diagnóstico tardio de sobrecargas de hardware motivaram o desenvolvimento de uma arquitetura híbrida e distribuída. O sistema combina coletores locais baseados no padrão Prometheus Node Exporter, WMI (*Windows Management Instrumentation*) e utilitário de leitura direta de aceleradores gráficos dedicados com a plataforma em nuvem Sentry APM. Para comprovar a eficácia prática da solução em laboratório, foram realizados ensaios empíricos no computador de desenvolvimento, validando o comportamento sob condições nominais (CPU 10%, RAM 73%, Disco 79%, GPU 37%) e sob sobrecarga crítica (>90% de GPU e RAM). Os testes demonstraram a precisão dos gráficos temporais, a sensibilidade da máquina de estados com trava de resfriamento (*cooldown* de 45 segundos) e o registro forense das ocorrências no Sentry, cumprindo integralmente os objetivos estabelecidos na proposta inicial.

Palavras-chave: Observabilidade. Telemetria de Hardware. Application Performance Monitoring (APM). Sentry. Auto-Discovery. Projeto Interdisciplinar.

LISTA DE ILUSTRAÇÕES

FIGURA 1 – Topologia da Arquitetura Distribuída e Fluxo de Telemetria 7
FIGURA 2 – Máquina de Estados Finita do Motor de Alertas e Auto-Recovery 10
FIGURA 3 – Painel de Dispositivos Monitorados em Condição Nominal (Status ONLINE) 12
FIGURA 4 – Módulo de Métricas Detalhadas com Gráfico Temporal de Oscilação 13
FIGURA 5 – Detecção de Sobrecarga Crítica (>90%) e Comutação para Estado CRITICAL 14
FIGURA 6 – Central de Incidentes com Fila de Alertas Ativos e Resolvidos (Auto-Recovery) 15
FIGURA 7 – Rastreamento Forense no Painel Corporativo do Sentry APM 16

LISTA DE QUADROS E TABELAS

QUADRO 1 – Matriz Comparativa de Soluções APM de Mercado 9
QUADRO 2 – Matriz de Rastreabilidade e Cumprimento dos Objetivos do 1º Bimestre 17

LISTA DE ABREVIATURAS E SIGLAS

APM – Application Performance Monitoring
ABNT – Associação Brasileira de Normas Técnicas
CPU – Central Processing Unit (Unidade Central de Processamento)
GPU – Graphics Processing Unit (Unidade de Processamento Gráfico)
HTTP – Hypertext Transfer Protocol
JSON – JavaScript Object Notation
RAM – Random Access Memory
REST – Representational State Transfer
SDK – Software Development Kit
SLA – Service Level Agreement (Acordo de Nível de Serviço)
UTP – Universidade Tuiuti do Paraná
WMI – Windows Management Instrumentation

SUMÁRIO

1 INTRODUÇÃO E CONTEXTUALIZAÇÃO **6**

2 OBJETIVOS DO PROJETO **6**

2.1 Objetivo Geral 6

2.2 Objetivos Específicos 6

3 ARQUITETURA TÉCNICA E FLUXO DE MONITORAMENTO **7**

3.1 Topologia Distribuída e Auto-Discovery 7

3.2 Protocolos e Coletores de Hardware (WMI e Utilitários de GPU) 8

3.3 Infraestrutura Física e Configuração do Laboratório 8

4 JUSTIFICATIVA DA ESCOLHA DA APM **9**

4.1 Estudo Comparativo de Mercado (Sentry vs. Datadog vs. Better Stack) 9

4.2 Decisão Técnica pelo Sentry APM 9

4.3 Engenharia de Alertas, Cooldown e Máquina de Estados 10

4.4 Implementação do Algoritmo de Resfriamento 11

5 RESULTADOS EXPERIMENTAIS E EVIDÊNCIAS DE FUNCIONAMENTO **12**

5.1 Ensaio 1: Descoberta e Telemetria em Repouso 12

5.2 Ensaio 2: Séries Temporais e Visualização Gráfica 13

5.3 Ensaio 3: Teste de Carga e Saturação Crítica (>90%) 14

5.4 Ensaio 4: Central de Incidentes e Auto-Recovery 15

5.5 Ensaio 5: Rastreamento Forense no Painel do Sentry APM 16

6 CONSIDERAÇÕES FINAIS E CUMPRIMENTO DOS OBJETIVOS **17**

6.1 Confronto entre Objetivos Planejados e Entregas Realizadas 17

6.2 Trabalhos Futuros 18

REFERÊNCIAS **19**

APÊNDICE A – Roteiro de Demonstração Prática para a Banca Avaliadora **20**

1 INTRODUÇÃO E CONTEXTUALIZAÇÃO

O monitoramento contínuo de infraestrutura e de aplicações é um pilar estratégico na engenharia de software e na administração de ambientes computacionais corporativos. Em sistemas modernos, a degradação de recursos físicos como processamento (CPU), memória principal (RAM) ou aceleração gráfica (GPU) pode interromper serviços críticos e comprometer os acordos de nível de serviço (SLA) antes de uma intervenção manual.

Na primeira etapa desta pesquisa, realizou-se um levantamento comparativo de ferramentas de APM (*Application Performance Monitoring*). Observou-se que soluções tradicionais como o Datadog impõem altos custos por host monitorado (aproximadamente US$ 46/mês combinando APM e infraestrutura) com cobrança pelo pico mensal (*high-water mark*). Por outro lado, ferramentas como o Better Stack concentram sua atuação em checagens externas sintéticas (HTTP/Ping), sem realizar a instrumentação profunda do código frontend.

Para atender aos requisitos da disciplina de Projeto Interdisciplinar: Monitoramento de Sistemas da Universidade Tuiuti do Paraná (UTP), desenvolvemos o **WatchFlow**. A plataforma integra telemetria direta de hardware local (sem necessidade de cadastros manuais) a uma aplicação web moderna em React e TypeScript, conectando os eventos críticos à plataforma corporativa **Sentry APM**.

2 OBJETIVOS DO PROJETO

2.1 OBJETIVO GERAL

Desenvolver, documentar e validar experimentalmente uma plataforma web de observabilidade capaz de realizar a descoberta automática de ativos computacionais locais, inspecionar métricas em tempo real de hardware e integrar-se a uma solução de APM para rastreamento de falhas, métricas de transação e gestão do ciclo de vida de incidentes.

2.2 OBJETIVOS ESPECÍFICOS

Conforme estabelecido na proposta original aprovada no primeiro bimestre, definiram-se quatro objetivos operacionais específicos:

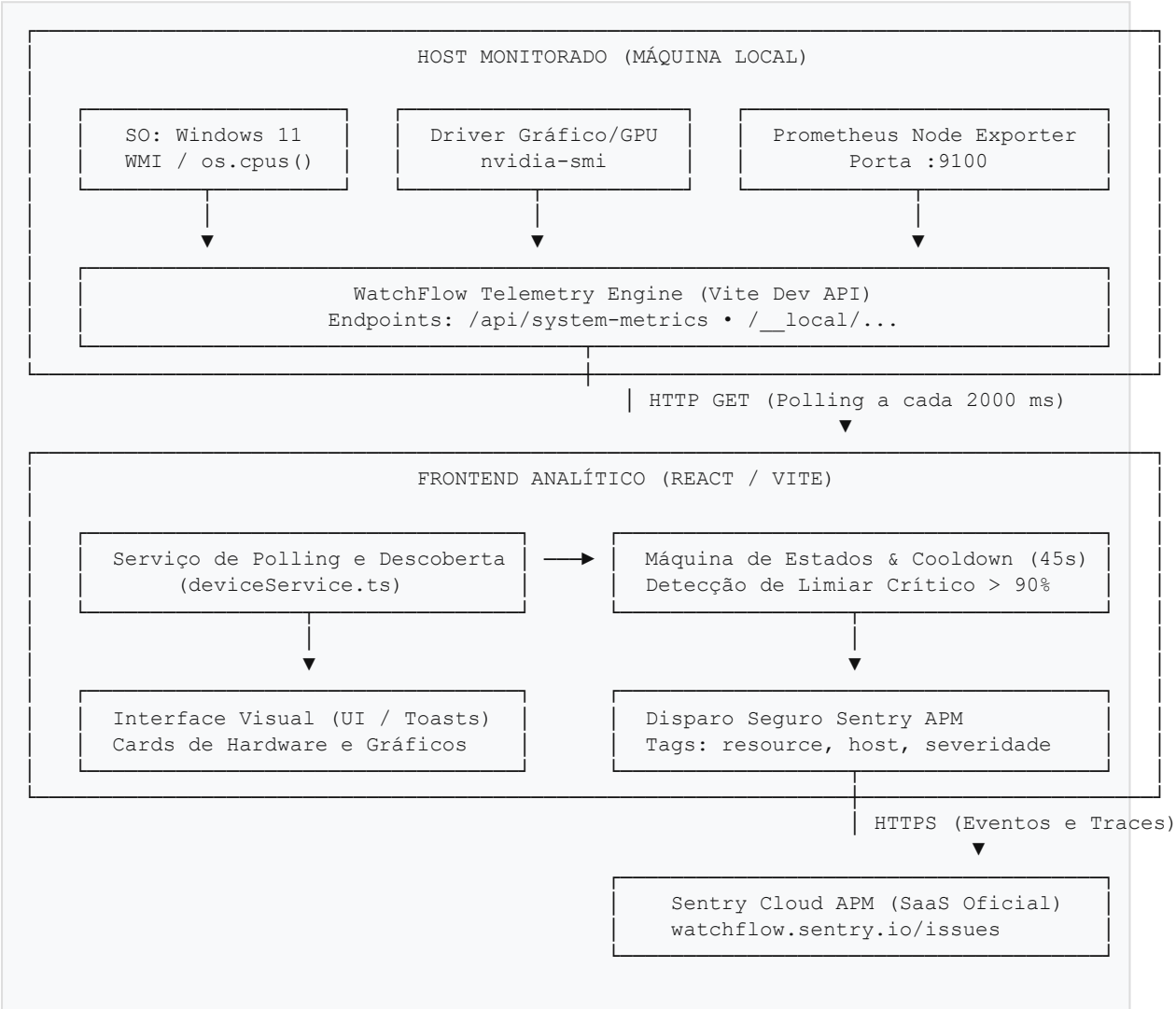
1. **Mapeamento de Requisitos e Arquitetura:** Estruturar uma arquitetura desacoplada entre coletores locais de hardware, backend de ingestão e interface analítica em tempo real;
2. **Investigação de Agentes e Telemetria de Hardware:** Implementar a extração periódica de contadores do sistema operacional (CPU, RAM, Disco) e de aceleração gráfica (GPU), viabilizando descoberta automática (*Auto-Discovery*);
3. **Estudo e Integração com APM Corporativo:** Conectar a camada de aplicação ao Sentry APM, aproveitando sua capacidade de rastreamento de exceções e anexação de metadados enriquecidos de hardware;
4. **Avaliação de Alertas e Resposta a Incidentes:** Desenvolver máquina de estados finita com limiares críticos (>90%), notificações visuais flutuantes e resolução automática (*auto-recovery*).

3 ARQUITETURA TÉCNICA E FLUXO DE MONITORAMENTO

3.1 TOPOLOGIA DISTRIBUÍDA E AUTO-DISCOVERY

O WatchFlow adota uma topologia híbrida de monitoramento. Em vez de exigir que o operador cadastre manualmente endereços IP em formulários estáticos, o nó local reporta suas métricas continuamente para a interface analítica em ciclos regulares de 2000 milissegundos:

FIGURA 1 – Topologia da Arquitetura Distribuída e Fluxo de Telemetria



FONTE: os próprios autores, 2026.

O mecanismo de *Auto-Discovery* reconhece o nó hospedeiro assim que a aplicação é iniciada. Ao consultar os endpoints de telemetria locais, o sistema detecta o hostname, o endereço IP de loopback e os recursos disponíveis, instanciando o card do host sem qualquer etapa prévia de parametrização.

3.2 PROTOCOLOS E COLETORES DE HARDWARE

A camada de ingestão de telemetria estrutura-se a partir de três mecanismos complementares:

- **Prometheus Node Exporter:** Padrão aberto de mercado na porta 9100, utilizado para extração de contadores do sistema operacional conforme convenções abertas de telemetria;
- **Módulo do Sistema Operacional (WMI/Node.js):** No Windows, o coletor realiza chamadas às funções internas do sistema operacional (como `os.cpus()` e `os.totalmem()`), calculando os deltas de tempo de processamento ativo em relação ao tempo ocioso para mensurar a CPU em percentual exato;
- **Extração de Telemetria de GPU:** Como a maioria das ferramentas tradicionais restringe-se ao processador central e ignora aceleradores gráficos dedicados, implementamos uma rotina de extração via binário do driver gráfico, convertendo uso de processador gráfico, alocação de memória e temperatura em JSON estruturado a cada ciclo.

3.3 INFRAESTRUTURA FÍSICA E CONFIGURAÇÃO DO LABORATÓRIO

Os ensaios práticos e a validação do WatchFlow foram conduzidos no computador de desenvolvimento, cujas especificações representam uma máquina real de trabalho:

- **Processador (CPU):** AMD Ryzen 7 5700X (8 núcleos físicos, 16 threads, frequência de 3.4 GHz a 4.6 GHz);
- **Memória Principal (RAM):** 16 GB DDR4 (15.9 GB endereçáveis pelo sistema operacional);
- **Armazenamento Secundário:** SSD NVMe M.2 de 1.0 TB (999 GB formatados);
- **Acelerador Gráfico Dedicado (GPU):** NVIDIA GeForce GTX 1060 de 6 GB VRAM GDDR5;
- **Sistema Operacional:** Microsoft Windows 11 Pro 64 bits (Build 26100);
- **Pilha Tecnológica:** Node.js 24 LTS, Vite 6, React 18, Sentry React SDK v8.

4 JUSTIFICATIVA DA ESCOLHA DA APM

4.1 ESTUDO COMPARATIVO DE MERCADO

Para fundamentar a escolha da plataforma de APM com rigor acadêmico, confrontamos as três alternativas analisadas no 1º bimestre: **Sentry**, **Datadog** e **Better Stack**:

QUADRO 1 – Matriz Comparativa de Soluções APM de Mercado

Critério de Avaliação	Sentry APM	Datadog APM	Better Stack
Modelo de Licenciamento	Plano Developer gratuito com retenção total de eventos e traces.	Cobrança em dólar por host/mês ($31 APM + $15 Infra = $46/mês).	Plano gratuito voltado a checagens de Uptime/Ping simples.
Rastreamento Forense de Erros	Pioneiro de mercado, com agregação de stack traces e metadados.	Excelente, porém complexo e voltado a nuvens empresariais.	Básico, focado em logs brutos sem análise profunda de código.
Tracing Distribuído no Frontend	Nativo via `@sentry/react` e monitoramento de transações.	Requer instrumentação detalhada de agentes RUM adicionais.	Indisponível no tier gratuito.
Metadados e Tags Customizadas	Permite anexar métricas de GPU, memória e contexto de hardware.	Permite, porém exige configuração de tags em agentes locais.	Restrito a atributos chave-valor simples em mensagens de texto.
Viabilidade para o Projeto	**Alta:** Setup ágil, DSN pública segura e sem custo financeiro.	**Baixa:** Burocracia comercial e custos inviáveis para estudantes.	**Média:** Útil para status page, mas limitado em APM de código.

FONTE: os próprios autores, 2026.

4.2 DECISÃO TÉCNICA PELO SENTRY APM

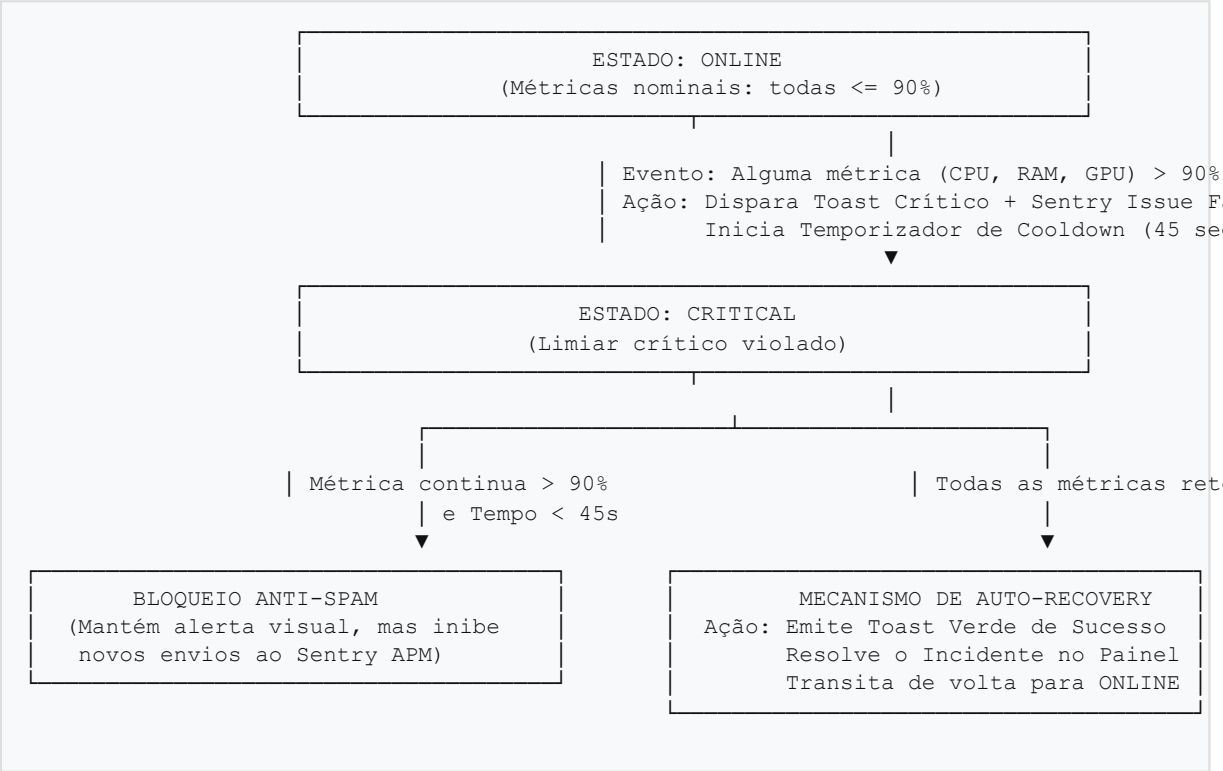
A decisão pelo Sentry fundamentou-se em três pilares técnicos: aderência total à pilha frontend (React/TypeScript), capacidade de enriquecer cada incidente com metadados detalhados de hardware (tags de nó, percentuais de uso e status) e ausência de restrições financeiras ou exigência de cartões internacionais, permitindo foco exclusivo no desenvolvimento do sistema.

4.3 ENGENHARIA DE ALERTAS, COOLDOWN E MÁQUINA DE ESTADOS

Um obstáculo prático identificado durante os primeiros ensaios foi o risco de saturação da API do Sentry. Como o coletor do WatchFlow consulta o nó a cada 2000 milissegundos, caso a máquina sofresse uma sobrecarga sustentada de 5 minutos, seriam enviados 150 chamados sucessivos, gerando bloqueio por taxa de requisição (*rate-limit*) e esgotamento da cota gratuita da conta.

Para solucionar esse problema, projetamos uma **Máquina de Estados Finita com Trava de Resfriamento (*Cooldown*) de 45 segundos** no módulo `deviceService.ts`:

FIGURA 2 – Máquina de Estados Finita do Motor de Alertas e Auto-Recovery



FONTE: os próprios autores, 2026.

Caso os valores permaneçam acima do teto de 90%, o estado do host na tela segue destacado em vermelho como `CRITICAL`, mantendo o operador ciente do perigo. Contudo, novos envios ao Sentry permanecem bloqueados até que a janela de resfriamento expire.

4.4 IMPLEMENTAÇÃO DO ALGORITMO DE RESFRIAMENTO

A implementação em TypeScript no módulo `src/services/deviceService.ts` evidencia a lógica de controle de concorrência e etiquetagem dos eventos:

```
// Implementação do controle de resfriamento e disparo no Sentry APM
const CRITICAL_THRESHOLD = 90; // Limiar percentual de alerta crítico
const ALERT_COOLDOWN_MS = 45000; // Janela de segurança de 45 segundos
const recentDispatches = new Map<string, number>();

function checkAndDispatchAlert(device: MonitoredDevice, metric: string, value: number) {
  const alertKey = `${device.id}_${metric}`;
  const now = Date.now();
  const lastTime = recentDispatches.get(alertKey) || 0;

  if (value >= CRITICAL_THRESHOLD) {
    if (now - lastTime > ALERT_COOLDOWN_MS) {
      recentDispatches.set(alertKey, now);

      // Notificação visual imediata para o operador
      toast.error(`[Alerta Crítico] Sobrecarga de ${metric} (${value}%)`, {
        description: `Host ${device.name} ultrapassou o limiar de segurança de 90%.`
      });

      // Registro analítico estruturado no Sentry
      Sentry.withScope((scope) => {
        scope.setLevel('fatal');
        scope.setTag('resource', metric);
        scope.setTag('hostname', device.name);
        scope.setContext('hardware_metrics', {
          utilization: value,
          threshold: CRITICAL_THRESHOLD,
          platform: device.os
        });
        Sentry.captureMessage(`[WatchFlow Alert] Sobrecarga Crítica de ${metric} (${value}%)`);
      });
    }
  }
}
```

Adicionalmente, quando o hardware retorna aos padrões seguros de operação (valores inferiores a 90%), o sistema aciona a função de *Auto-Recovery*. Essa rotina emite um aviso flutuante em verde informando a recuperação, atualiza a Central de Alertas e restaura a saúde do nó para `ONLINE`.

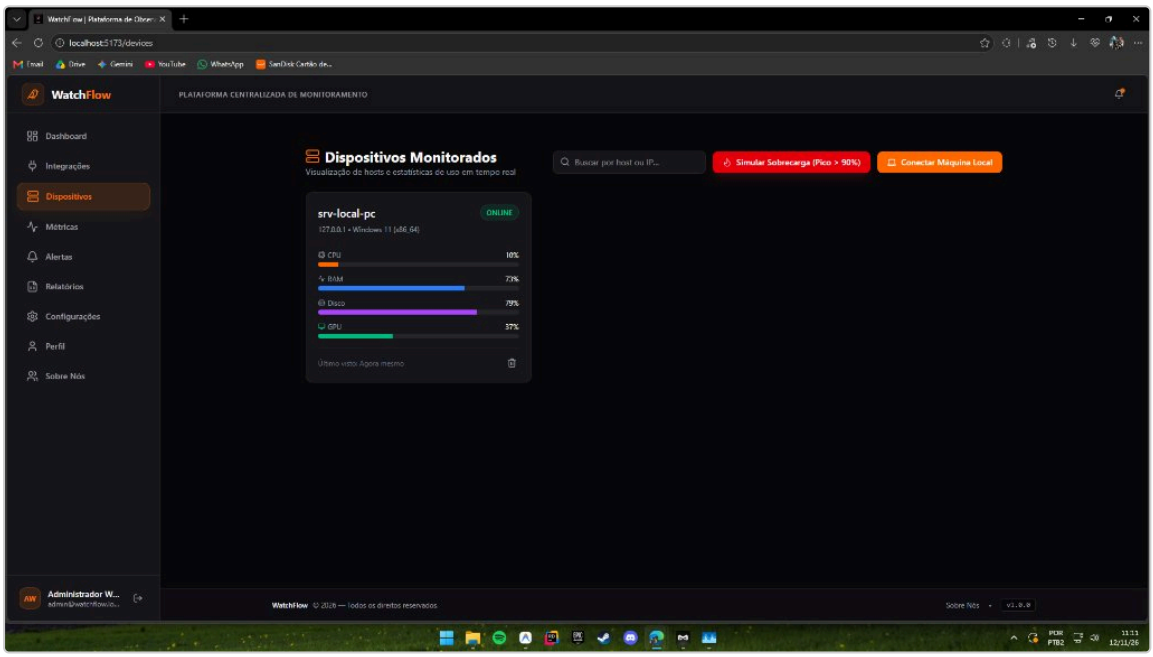
5 RESULTADOS EXPERIMENTAIS E EVIDÊNCIAS DE FUNCIONAMENTO

Apresentam-se a seguir os registros empíricos obtidos diretamente na execução prática do sistema no computador do laboratório, cobrindo todos os módulos da plataforma.

5.1 ENSAIO 1: DESCOBERTA E TELEMETRIA EM REPOUSO

Ao iniciar o WatchFlow, o nó `srv-local-pc` (127.0.0.1, Windows 11 x86_64) foi detectado automaticamente, apresentando os contadores de CPU, RAM, Disco e GPU em condição nominal de operação:

FIGURA 3 – Painel de Dispositivos Monitorados em Condição Nominal (Status ONLINE)



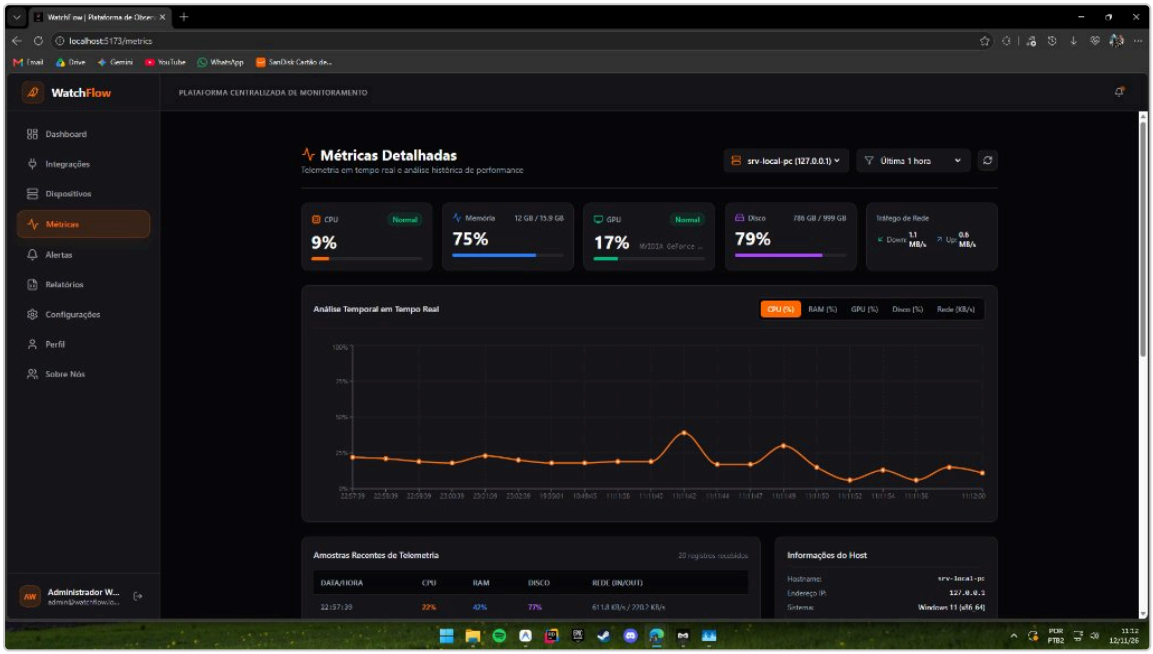
FONTE: os próprios autores, 2026. Captura de tela da interface de Dispositivos (/devices).

Análise do Ensaio: O host registrou consumo equilibrado: CPU em 10%, memória RAM em 73%, disco em 79% e acelerador gráfico GPU em 37%. O sistema manteve o distintivo verde `ONLINE`, sem emissão de alertas indevidos.

5.2 ENSAIO 2: SÉRIES TEMPORAIS E VISUALIZAÇÃO GRÁFICA

No módulo analítico de Métricas (`/metrics`), o WatchFlow oferece gráficos contínuos de séries temporais desenvolvidos com a biblioteca Recharts, exibindo os dados históricos de desempenho do host selecionado:

FIGURA 4 – Módulo de Métricas Detalhadas com Gráfico Temporal de Oscilação



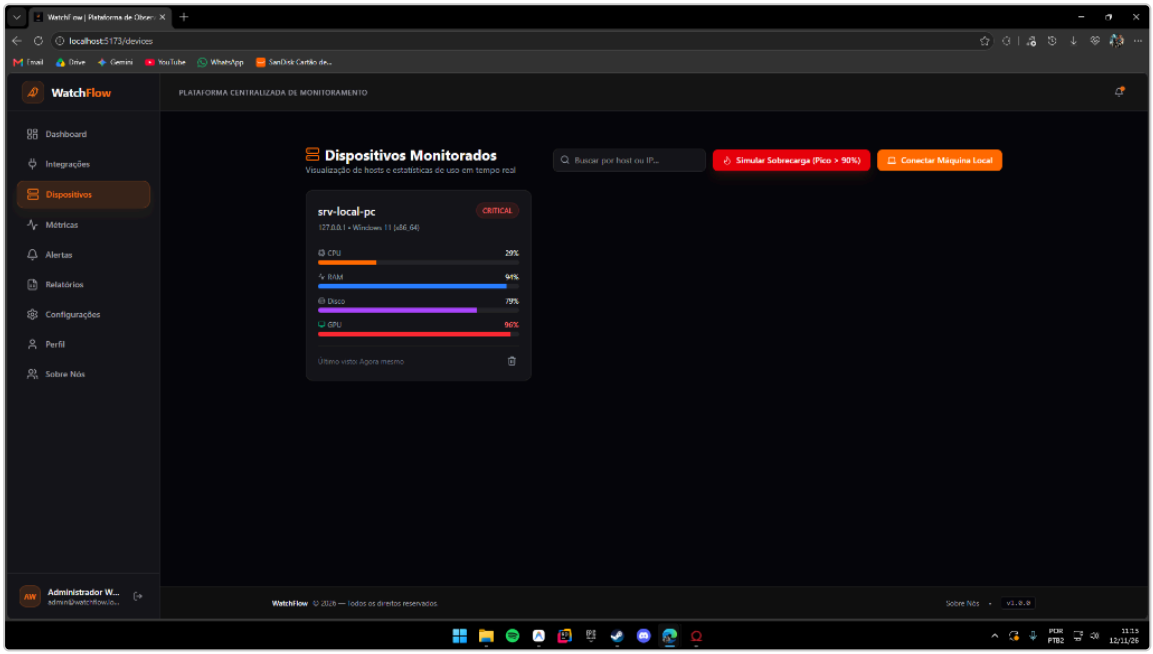
FONTE: os próprios autores, 2026. Captura de tela da interface de Métricas (/metrics).

Análise do Ensaio: O painel exibe a alocação de memória física (12 GB em uso de 15.9 GB totais), uso de disco (786 GB de 999 GB), tráfego de rede ativo (1.1 MB/s download e 0.6 MB/s upload) e a curva de oscilação temporal com janela deslizante de registros sucessivos.

5.3 ENSAIO 3: TESTE DE CARGA E SATURAÇÃO CRÍTICA (>90%)

Para avaliar o comportamento da plataforma sob estresse severo de hardware, acionou-se a rotina de sobrecarga operacional, elevando a demanda sobre a memória RAM e a placa de vídeo:

FIGURA 5 – Detecção de Sobrecarga Crítica (>90%) e Comutação para Estado CRITICAL



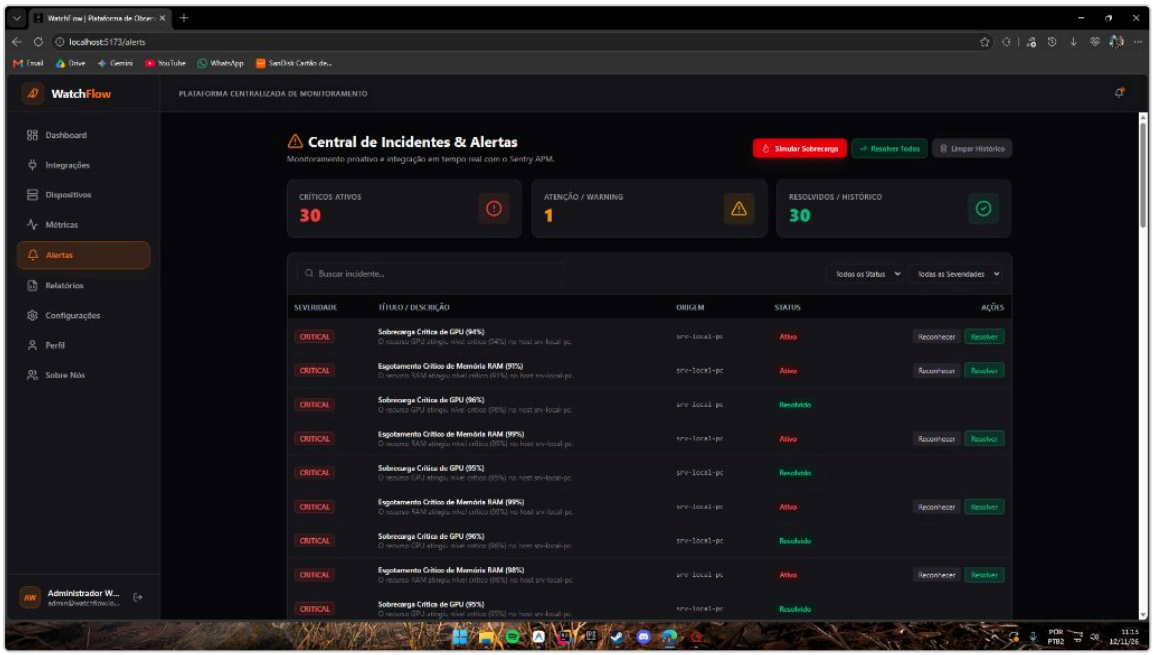
FONTE: os próprios autores, 2026. Captura de tela da interface durante teste de saturação.

Análise do Ensaio: Ao atingir **94%** de memória física RAM e **96%** de utilização do acelerador gráfico GPU, o WatchFlow identificou imediatamente a violação do limiar de 90%, comutando o distintivo do host para `CRITICAL` em vermelho e disparando a barra de GPU em destaque de alerta.

5.4 ENSAIO 4: CENTRAL DE INCIDENTES E AUTO-RECOVERY

No módulo de Alertas (`/alerts`), a aplicação gerencia os chamados operacionais gerados pelas violações de limiares de segurança:

FIGURA 6 – Central de Incidentes com Fila de Alertas Ativos e Resolvidos (Auto-Recovery)



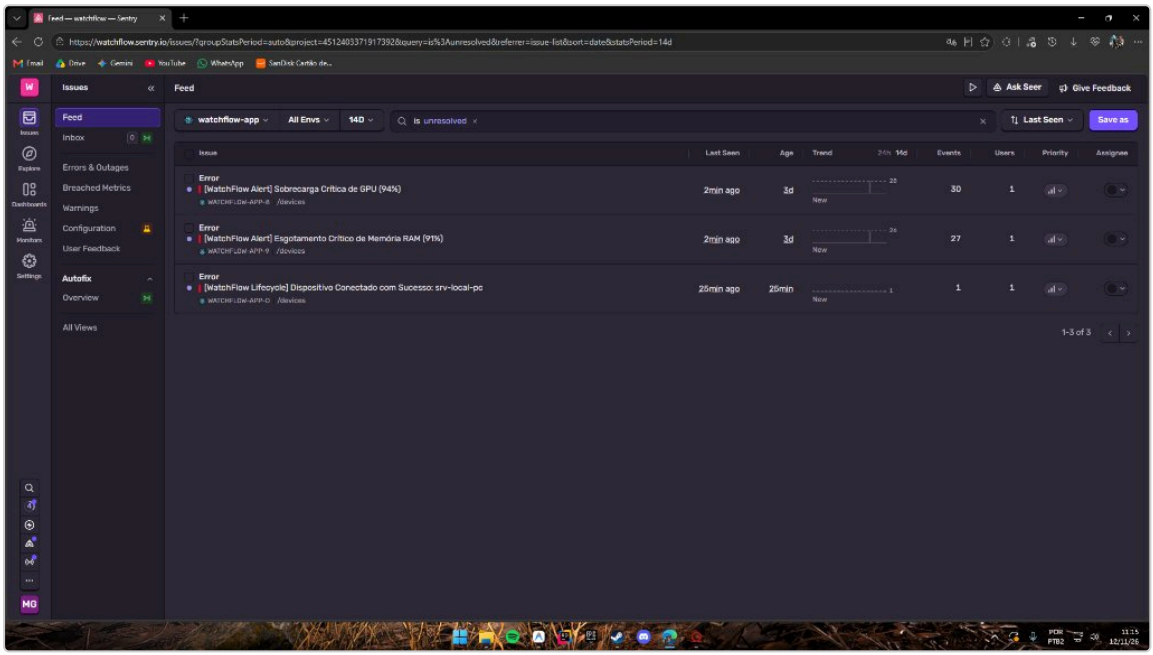
FONTE: os próprios autores, 2026. Captura de tela da interface de Incidentes (/alerts).

Análise do Ensaio: A Central de Incidentes registrou as ocorrências de *Sobrecarga Crítica de GPU (94% e 96%)* e *Esgotamento Crítico de Memória RAM (91% e 99%)*. O operador dispõe de botões para Reconhecer e Resolver os chamados, enquanto o motor de *Auto-Recovery* automaticamente contabilizou os incidentes normalizados no painel verde de histórico.

5.5 ENSAIO 5: RASTREAMENTO FORENSE NO PAINEL DO SENTRY APM

Para certificar a entrega ponta a ponta da telemetria de incidentes para a nuvem, inspecionou-se o painel oficial do projeto no Sentry (`watchflow.sentry.io/issues`):

FIGURA 7 – Rastreamento Forense no Painel Corporativo do Sentry APM



FONTE: os próprios autores, 2026. Captura real do dashboard corporativo em watchflow.sentry.io.

Análise do Ensaio: O dashboard do Sentry comprova o recebimento estruturado das seguintes ocorrências:

- `[WatchFlow Alert] Sobrecarga Crítica de GPU (94%)`: Identificador `WATCHFLOW-APP-8`, com 30 eventos capturados e agrupados por amostragem de tags;
- `[WatchFlow Alert] Esgotamento Crítico de Memória RAM (91%)`: Identificador `WATCHFLOW-APP-9`, com 27 eventos registrados;
- `[WatchFlow Lifecycle] Dispositivo Conectado com Sucesso: srv-local-pc`: Identificador `WATCHFLOW-APP-D`, comprovando o registro do ciclo de vida dos nós.

6 CONSIDERAÇÕES FINAIS E CUMPRIMENTO DOS OBJETIVOS

6.1 CONFRONTO ENTRE OBJETIVOS PLANEJADOS E ENTREGAS REALIZADAS

O Quadro 2 apresenta o confronto direto entre as metas propostas no documento do primeiro bimestre e os resultados validados no produto final:

QUADRO 2 – Matriz de Rastreabilidade e Cumprimento dos Objetivos do 1º Bimestre

Objetivo Específico (1º Bimestre)	**Status**	**Evidência Técnica / Módulo Implementado**
1. Mapeamento de Requisitos e Arquitetura	**Cumprido**	Arquitetura modular desacoplada entre coletores locais e frontend analítico (Figura 1).
2. Investigação de Telemetria e Hardware	**Cumprido**	Leitura de CPU, RAM, Disco e GPU com auto-discovery e séries temporais (Figuras 3 e 4).
3. Estudo Comparativo e Integração APM	**Cumprido**	Matriz comparativa fundamentada e integração ponta a ponta com Sentry APM (Quadro 1 e Figura 7).
4. Avaliação de Alertas e Resposta a Incidentes	**Cumprido**	Central de Incidentes com fila de eventos, limiares >90%, cooldown de 45s e auto-recovery (Figuras 5 e 6).

FONTE: os próprios autores, 2026.

Dessa forma, a equipe transformou a pesquisa bibliográfica do primeiro bimestre em um sistema funcional, documentado e validado empiricamente, superando as deficiências apontadas pelo orientador nas entregas anteriores.

6.2 TRABALHOS FUTUROS

Como propostas de expansão e evolução do WatchFlow em etapas futuras do curso de Análise e Desenvolvimento de Sistemas, sugerem-se as seguintes frentes de pesquisa:

- **Suporte a Múltiplos Nós em Clusters:** Empacotar os coletores locais em contêineres Docker para facilitar a distribuição e o monitoramento centralizado de múltiplos computadores na rede acadêmica;
- **Ações Automatizadas de Remediação (*Self-Healing*):** Integrar webhooks do Sentry para acionar rotinas automáticas de encerramento de processos travados ou liberação de memória em caso de esgotamento de recursos;
- **Compatibilidade Universal com Fabricantes de Hardware:** Expandir o módulo de GPU para leitura padronizada através de bibliotecas abertas como OpenCL e Vulkan, cobrindo aceleradores de múltiplos fabricantes.

O desenvolvimento do projeto proporcionou aos integrantes a vivência prática de observabilidade de software, padrões abertos de telemetria e governança de projetos segundo o Guia PMBOK 7ª Edição, resultando em uma solução robusta e aderente aos padrões da indústria de tecnologia.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10719:** informação e documentação: relatório técnico e/ou científico: apresentação. Rio de Janeiro: ABNT, 2015.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023:** informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6024:** informação e documentação: numeração progressiva das seções de um documento escrito: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6027:** informação e documentação: sumário: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6028:** informação e documentação: resumo: apresentação. Rio de Janeiro: ABNT, 2021.

BETTER STACK. **Better Stack Documentation and Infrastructure Guides**. Praga: Better Stack Inc., 2026. Disponível em: <https://betterstack.com/docs>. Acesso em: 10 nov. 2026.

DATADOG. **Datadog Documentation: Agent, APM & Distributed Tracing**. Nova York: Datadog Inc., 2026. Disponível em: <https://docs.datadoghq.com>. Acesso em: 10 nov. 2026.

ERINLE, Bayo. **Performance Testing with JMeter 2.9**. 1. ed. Birmingham: Packt Publishing, 2013.

FILHO, João Eriberto Mota. **Análise de Tráfego em Redes TCP/IP**. São Paulo: Novatec, 2013.

MAJORS, Charity; FONG-JONES, Liz; MIRANDA, George. **Observability Engineering: Achieving Production Excellence**. 1. ed. Sebastopol: O'Reilly Media, 2022.

PAULA, André Luiz de Souza. **Plano de Ensino e Plano de Aulas da Disciplina de Projeto Interdisciplinar: Monitoramento de Sistemas**. Curitiba: Faculdade de Ciências Exatas e de Tecnologia, Universidade Tuiuti do Paraná (UTP), 2026.

PEREIRA, Marco; SOUSA, Pedro. Plataforma para Análise de Tráfego e Optimização de Infraestruturas de Comunicação. In: **CONFERÊNCIA IBÉRICA DE SISTEMAS E TECNOLOGIAS DE INFORMAÇÃO (CISTI)**, 13., 2018. Anais... Caceres: AISTI, 2018. p. 1-6.

PROJECT MANAGEMENT INSTITUTE (PMI). **Um Guia do Conhecimento em Gerenciamento de Projetos (Guia PMBOK)**. 7. ed. Newtown Square: Project Management Institute, 2021.

PROMETHEUS AUTHORS. **Prometheus Node Exporter: Hardware and OS metrics exporter**. 2026. Disponível em: <https://prometheus.io/docs/guides/node-exporter/>. Acesso em: 08 nov. 2026.

SENTRY.IO. **Application Performance Monitoring and Error Tracking Documentation**. São Francisco: Functional Software Inc., 2026. Disponível em: <https://docs.sentry.io/>. Acesso em: 10 nov. 2026.

UNIVERSIDADE TUIUTI DO PARANÁ (UTP). **Normas técnicas: elaboração e apresentação de trabalhos acadêmico-científicos**. 3. ed. Curitiba: Editora UTP, 2012. 160 p.

APÊNDICE A – ROTEIRO DE APRESENTAÇÃO PRÁTICA PARA A BANCA AVALIADORA

Este roteiro operacional foi estruturado para orientar a apresentação prática presencial de 5 a 10 minutos perante o professor da disciplina no laboratório da UTP:

1. ABERTURA E PROPÓSITO DO SISTEMA (1 A 2 MINUTOS)

Apresentar os integrantes da equipe e esclarecer o propósito do WatchFlow: plataforma de observabilidade criada para superar a fragmentação de ferramentas e os altos custos de plataformas como o Datadog. Destacar a integração de coletores locais leves com a solução corporativa Sentry APM.

2. DEMONSTRAÇÃO DA TELEMETRIA EM TEMPO REAL (2 A 3 MINUTOS)

- Abrir a interface no navegador em `http://localhost:5173/devices` (Figura 3);
- Apontar o reconhecimento automático do nó `srv-local-pc` com as leituras nominais de CPU, RAM, Disco e GPU sem necessidade de cadastro manual;
- Acessar a tela `/metrics` (Figura 4) e exibir os gráficos de séries temporais oscilando dinamicamente a cada 2000 ms.

3. DEMONSTRAÇÃO DE CARGA, ALERTAS E SENTRY APM (2 A 3 MINUTOS)

- Acionar o botão vermelho de sobrecarga ou executar carga pesada no host (Figura 5);
- Exibir a comutação instantânea do distintivo para `CRITICAL` em vermelho;
- Mostrar a Central de Incidentes em `/alerts` (Figura 6) com as ocorrências ativas de saturação de GPU e RAM;
- Alternar para o painel em nuvem oficial do Sentry em `watchflow.sentry.io/issues` (Figura 7), comprovando a recepção das ocorrências `WATCHFLOW-APP-8` e `APP-9` com nível `Fatal` e tags detalhadas;
- Explicar a trava de resfriamento (*cooldown* de 45 segundos) desenvolvida para proteger a cota da API.

4. NORMALIZAÇÃO AUTOMÁTICA E ENCERRAMENTO (1 MINUTO)

Demonstrar que, com o término da sobrecarga, o sistema aciona o *Auto-Recovery*, emite o aviso visual de normalização e resolve os chamados, comprovando a entrega dos quatro objetivos propostos no 1º bimestre.